Cẩm nang triển khai LLM tự host cho hệ thống VISTA

Kiến thức nền tảng Kiến trúc RAG Bảo mật Vận hành Lộ trình triển khai

Phiên bản 1.0 | Tháng 10 năm 2026

Tài liệu này hướng dẫn đội VISTA chuyển từ việc phụ thuộc trực tiếp vào Gemini API sang một nền tảng AI do dự án tự kiểm soát. Kết luận chính là: không huấn luyện một mô hình ngôn ngữ lớn từ đầu; hãy tự host mô hình open weight, bổ sung dữ liệu VISTA bằng RAG, giữ toàn bộ luật nghiệp vụ trong Spring Boot và xây lớp AI Provider để có thể đổi mô hình mà không viết lại sản phẩm.

Đối tượng đọc gồm thành viên backend, mobile, web, DevOps, kiểm thử và người quản lý kỹ thuật. Sau khi đọc, nhóm có thể chọn cấu hình phần cứng, dựng môi trường thử nghiệm, tích hợp model server, xây kho tri thức, bảo vệ dữ liệu, đánh giá chất lượng và chuyển đổi dần khỏi Gemini mà không làm gián đoạn dịch vụ.

Mục lục nội dung

1 Điều cần hiểu trước khi bắt đầu

2 LLM hoạt động như thế nào

3 Các cách xây AI và lựa chọn phù hợp

4 Kiến trúc mục tiêu cho VISTA

5 Lựa chọn mô hình và runtime

6 Lựa chọn phần cứng

7 RAG và kho tri thức du lịch

8 Embedding và tìm kiếm hybrid

9 Tool calling và luật nghiệp vụ

10 Structured output và kiểm chứng

11 Thiết kế AI service

12 Tích hợp Spring Boot

13 Hàng đợi và streaming

14 Bảo mật và quyền riêng tư

15 Quan sát và kiểm soát chi phí

16 Bộ đánh giá chất lượng

17 Fine tuning và LoRA

18 Quy trình triển khai từng giai đoạn

19 Runbook vận hành

20 Checklist nghiệm thu

1 Điều cần hiểu trước khi bắt đầu

1.1 Mục tiêu thực tế

Mục tiêu của VISTA không phải là tạo một mô hình nền tảng mới cạnh tranh với các hãng AI. Mục tiêu là sở hữu chuỗi xử lý AI: chọn model, tự vận hành inference, kiểm soát dữ liệu, kiểm soát quota, gắn kiến thức du lịch của VISTA và thay model khi cần.

Một hệ thống AI production gồm nhiều thành phần hơn model. Model chỉ sinh token. Chất lượng sản phẩm còn phụ thuộc vào dữ liệu truy xuất, prompt, validation, công cụ nghiệp vụ, bảo mật, quan sát, giao diện và quy trình xử lý lỗi.

1.2 Bốn khái niệm thường bị nhầm

Khái niệm	Ý nghĩa	VISTA nên dùng khi nào
Pretraining	Huấn luyện hàng tỷ tham số từ kho dữ liệu cực lớn.	Không phù hợp cho dự án hiện tại.

Khái niệm	Ý nghĩa	VISTA nên dùng khi nào
Fine tuning	Điều chỉnh hành vi model bằng tập ví dụ có nhãn.	Chỉ cân nhắc sau khi có dữ liệu đánh giá tốt.
RAG	Tìm dữ liệu liên quan rồi đưa vào context trước khi sinh câu trả lời.	Là nền tảng chính để dùng dữ liệu du lịch VISTA.
Prompt engineering	Thiết kế instruction, schema và ví dụ cho model.	Dùng ngay, nhưng phải có version và test.

1.3 Nguyên tắc nguồn sự thật

- LLM không phải nguồn sự thật cho giá, giờ mở cửa, số dư, quyền truy cập hoặc trạng thái thanh toán.
- PostgreSQL và các service nghiệp vụ là nguồn sự thật.
- LLM chỉ đề xuất; backend xác minh rồi mới lưu.
- Khi không có dữ liệu đáng tin cậy, hệ thống phải nói không biết thay vì bịa.

2 LLM hoạt động như thế nào

2.1 Token và context

LLM nhận văn bản đã được chia thành token và dự đoán token tiếp theo. Context window là tổng số token model có thể xử lý trong một request, bao gồm system prompt, lịch sử, dữ liệu RAG, tool result và output. Context dài làm tăng bộ nhớ KV cache và độ trễ; vì vậy không nên đẩy toàn bộ database vào prompt.

2.2 Tham số và lượng tử hóa

Số tham số phản ánh kích thước model, không đảm bảo trực tiếp chất lượng cho mọi nhiệm vụ. Quantization chuyển trọng số sang độ chính xác thấp hơn như 8 bit hoặc 4 bit để giảm VRAM và tăng khả năng chạy trên phần cứng nhỏ. Đổi lại, chất lượng hoặc tốc độ có thể suy giảm tùy model và runtime.

Dạng	Ưu điểm	Hạn chế
BF16 hoặc FP16	Chất lượng ổn định, hỗ trợ tốt trên GPU server.	Tốn VRAM.
INT8	Giảm bộ nhớ, thường giữ chất lượng khá tốt.	Không phải runtime nào cũng tối ưu như nhau.
INT4 AWQ hoặc GPTQ	Phù hợp GPU 16 đến 24 GB.	Cần benchmark chất lượng và kernel.
GGUF Q4 hoặc Q5	Dễ chạy với llama.cpp trên CPU hoặc GPU cá nhân.	Thông lượng production thường thấp hơn cấu hình GPU chuyên dụng.

2.3 Sampling

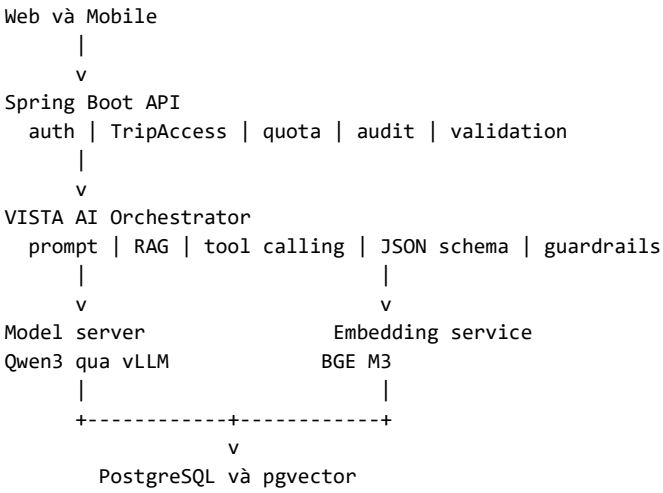
Temperature, top p và max tokens ảnh hưởng tính ngẫu nhiên. Với lập lịch trình có JSON schema, nên dùng temperature thấp để ổn định. Không dùng temperature cao để sửa vấn đề thiếu dữ liệu; điều đó chỉ làm câu trả lời đa dạng hơn, không đáng tin hơn.

3 Các cách xây AI và lựa chọn phù hợp

Phương án	Chi phí đầu tư	Kiểm soát	Khuyến nghị
API bên ngoài	Thấp lúc đầu	Thấp đến trung bình	Giữ làm fallback tạm thời.
Self host open weight	Trung bình	Cao	Phương án mục tiêu cho VISTA.
Fine tune model có sẵn	Trung bình đến cao	Cao	Thực hiện sau RAG và evaluation.
Pretrain từ đầu	Rất cao	Rất cao	Không triển khai ở giai đoạn này.

Quyết định đề xuất là triển khai self hosted inference với interface tương thích OpenAI, để backend có thể gọi model local, model cloud hoặc Gemini qua cùng một abstraction. Điều này tránh khóa nhà cung cấp và cho phép chuyển traffic từng phần.

4 Kiến trúc mục tiêu cho VISTA



4.1 Phân chia trách nhiệm

Thành phần	Chịu trách nhiệm	Không được làm
Web và Mobile	Thu thập yêu cầu, hiển thị tiến trình, lỗi và citations.	Không chứa API key hoặc tự xác nhận dữ liệu AI.
Spring Boot	Auth, quyền chuyến đi, quota, transaction, validation, lưu dữ liệu.	Không nhúng model runtime vào cùng JVM.
AI Orchestrator	RAG, prompt, gọi model, schema, tool loop.	Không truy cập database bằng SQL tùy ý.
Model server	Inference và batching.	Không expose trực tiếp ra Internet.
PostgreSQL	Nguồn dữ liệu thật và vector store.	Không cho model quyền database trực tiếp.

4.2 Luồng tạo lịch trình

1. Client gửi yêu cầu tới Spring Boot.
2. Spring Boot xác thực user và quyền chuyến đi.
3. Backend chuẩn hóa ngân sách, số ngày và sở thích.
4. Orchestrator truy xuất destination PUBLISHED bằng RAG.
5. Model lập phương án từ danh sách được phép.
6. Backend tính lại chi phí và kiểm tra mọi destination ID.
7. Nếu hợp lệ, trả preview; người dùng xác nhận mới lưu.

5 Lựa chọn mô hình và runtime

5.1 Model khởi đầu

Khuyến nghị bắt đầu với Qwen3 8B vì kích thước vừa phải, hỗ trợ tiếng Việt, có giấy phép Apache 2.0 và được hỗ trợ bởi các runtime phổ biến. Nếu bộ đánh giá cho thấy 8B chưa đạt, thử Qwen3 14B trước khi nhảy lên model rất lớn.

Môi trường	Model	Runtime	Mục tiêu
Laptop CPU hoặc GPU nhỏ	Qwen3 4B hoặc 8B GGUF Q4	llama.cpp	Phát triển chức năng.
GPU 16 đến 24 GB	Qwen3 8B quantized	vLLM	MVP production ít tải.
GPU 24 đến 48 GB	Qwen3 14B quantized	vLLM	Tăng chất lượng.
Nhiều GPU	Qwen3 32B hoặc model thắng evaluation	vLLM distributed	Scale sau khi có tải thật.

5.2 llama.cpp và vLLM

llama.cpp phù hợp môi trường phát triển vì có thể chạy GGUF trên CPU, CUDA, Vulkan và kết hợp CPU GPU. vLLM phù hợp server GPU vì có continuous batching, quản lý KV cache và parallelism tốt hơn. Cả hai có API tương thích OpenAI, giúp giữ nguyên contract giữa backend và model server.

5.3 Quy trình chọn model

- Pin chính xác model revision và checksum.
- Đọc giấy phép model và điều kiện phân phối.
- Chạy cùng một bộ prompt tiếng Việt trên từng ứng viên.
- Đo schema pass rate, hallucination, budget violation, latency và VRAM.
- Chọn model nhỏ nhất đạt ngưỡng thay vì model lớn nhất có thể chạy.

6 Lựa chọn phần cứng

6.1 Ước lượng bộ nhớ

Bộ nhớ inference không chỉ chứa weights. Nó còn chứa KV cache, activation, runtime overhead và batch đang chạy. Một model vừa VRAM ở batch 1 chưa chắc chịu được nhiều người dùng hoặc context dài.

Yếu tố	Làm tăng tài nguyên	Cách kiểm soát
Model size	Nhiều tham số hơn.	Chọn model nhỏ hơn hoặc quantization.
Context length	KV cache tăng theo token và concurrency.	Giới hạn context, RAG gọn và truncate có chủ đích.
Concurrency	Nhiều sequence đồng thời.	Queue, admission control và batching.
Output length	GPU bị giữ lâu hơn.	Đặt max tokens theo use case.
Embedding jobs	Dùng CPU hoặc GPU song song.	Tách service và chạy batch ngoài giờ cao điểm.

6.2 Không mua GPU trước benchmark

Hãy đo request mỗi ngày, concurrency đỉnh, context trung bình, output trung bình, time to first token và tokens mỗi giây. Sau đó thuê GPU theo giờ để benchmark trước khi mua. GPU 24 GB thường là điểm bắt đầu hợp lý cho model 8B, nhưng đây không phải bảo đảm; kết quả phụ thuộc quantization, context và tải đồng thời.

7 RAG và kho tri thức du lịch

7.1 Vì sao VISTA cần RAG

Giá vé, giờ mở cửa và trạng thái địa điểm thay đổi. Fine tuning không phải cơ chế cập nhật dữ liệu thời gian thực. RAG cho phép model nhận đúng dữ liệu mới nhất ở mỗi request và trả kèm nguồn.

7.2 Pipeline ingestion

8. Đọc bản ghi destination và tài liệu đã được duyệt.
9. Chuẩn hóa Unicode, đơn vị tiền, thời gian và địa danh.
10. Tách chunk theo ngữ nghĩa, không cắt giữa trường quan trọng.
11. Gắn metadata và content hash.
12. Tạo embedding theo batch.
13. Upsert chunk và embedding.
14. Đánh dấu lại khi source thay đổi hoặc bị unpublish.

7.3 Schema dữ liệu

```
ai_knowledge_documents
  id, source_type, source_id, title, content_hash, language, status, updated_at

ai_knowledge_chunks
  id, document_id, chunk_index, content, metadata_json, embedding, embedding_model, created_at
```

7.4 Quy tắc retrieval

- Chỉ truy xuất destination PUBLISHED.
- Áp dụng filter tỉnh, vùng, category, price và accessibility trước vector ranking.

- Dùng hybrid search thay vì chỉ vector similarity.
- Rerank top candidates rồi mới đưa vào prompt.
- Mọi citation phải trỏ về source ID và thời điểm cập nhật.

8 Embedding và tìm kiếm hybrid

BGE M3 là ứng viên phù hợp vì hỗ trợ đa ngôn ngữ, dense retrieval, sparse retrieval và input dài. Tuy nhiên vẫn cần benchmark riêng với truy vấn tiếng Việt của VISTA.

8.1 Hybrid retrieval

```
vector_score = cosine_similarity(query_embedding, chunk_embedding)
keyword_score = PostgreSQL full text rank
final_score = reciprocal_rank_fusion(vector_rank, keyword_rank)
result = rerank(top_candidates)
```

Vector search tìm nội dung gần nghĩa; lexical search giữ tốt tên riêng như Bà Nà, Hội An hoặc mã địa điểm. Kết hợp hai loại giảm bỏ sót và giảm kết quả gần nghĩa nhưng sai địa danh.

8.2 Index và cập nhật

- Dùng pgvector cùng PostgreSQL hiện tại để giảm số hệ thống phải vận hành.
- Chỉ thêm HNSW khi dữ liệu và tải đủ lớn; dữ liệu nhỏ có thể exact search tốt hơn.
- Tạo index concurrently ở production.
- Lưu embedding model và version để reindex có kiểm soát.

9 Tool calling và luật nghiệp vụ

Tool calling biến LLM thành bộ điều phối có giới hạn. Model được phép yêu cầu dữ liệu hoặc phép tính qua tool, nhưng tool do hệ thống kiểm soát và validate.

Tool	Đầu vào	Đầu ra tin cậy
search_destinations	Tỉnh, category, budget, accessibility	Danh sách ID PUBLISHED có metadata.
get_destination_details	Destination ID	Giá, giờ mở cửa, tọa độ, nguồn.
calculate_route	Danh sách tọa độ đã kiểm tra	Khoảng cách, thời lượng và route.
calculate_budget	Activity và thành viên	BigDecimal totals do backend tính.
get_trip_constraints	Trip ID từ context bảo mật	Ngày, thành viên, giới hạn của chuyến đi.

Không cung cấp tool chạy SQL, shell hoặc gọi URL tùy ý. Tool phải có allowlist, schema, timeout, authorization và audit.

10 Structured output và kiểm chứng

10.1 JSON Schema

```
{
  "type": "object",
```

```
  "required": ["days", "estimatedTotalVnd"],
  "properties": {
    "days": {"type": "array", "minItems": 1, "maxItems": 7},
    "estimatedTotalVnd": {"type": "integer", "minimum": 0}
  },
  "additionalProperties": false
}
```

10.2 Hai tầng validation

- Tầng cú pháp kiểm tra JSON Schema, type, enum, độ dài và số phần tử.
- Tầng nghiệp vụ kiểm tra destination tồn tại, số ngày, giờ, tọa độ, ngân sách và quyền truy cập.

Nếu output lỗi, chỉ cho phép một lần repair có giới hạn. Nếu vẫn lỗi, trả trạng thái FAILED rõ ràng. Không dựng dữ liệu mẫu rồi báo thành công.

11 Thiết kế AI service

11.1 Cấu trúc đề xuất

```
ai-service/
  app/
    api/
    inference/
    retrieval/
    prompts/
    schemas/
    tools/
    guardrails/
    evaluation/
  tests/
  Dockerfile
  requirements.lock
```

11.2 API nội bộ

```
POST /internal/v1/itineraries/generate
Authorization: Bearer <service-token>
X-Request-ID: <uuid>
Idempotency-Key: <uuid>
```

API nội bộ không được public. Spring Boot truyền dữ liệu đã chuẩn hóa và không truyền JWT người dùng cho model service. Service token phải được xoay vòng và lưu bằng secret manager.

11.3 Response có khả năng truy vết

```
{
  "model": "qwen3-8b",
  "modelRevision": "pinned-revision",
  "promptVersion": "itinerary-v1",
  "retrievalVersion": "destinations-v1",
  "days": [],
  "citations": [],
  "warnings": []
}
```

12 Tích hợp Spring Boot

12.1 Provider abstraction

```
public interface AiProvider {
  AiPlanResult generate(AiPlanCommand command);
  boolean isHealthy();
  String providerName();
}
```

Tạo LocalLlmProvider và giữ GeminiProvider trong thời gian chuyển đổi. AiProviderRouter chọn provider bằng feature flag, health, quota và rollout percentage. Không đặt fallback âm thầm; response phải ghi rõ provider và fallback status cho telemetry.

12.2 Luồng service

15. CurrentUserProvider lấy danh tính.
16. TripAccessService xác minh thành viên.
17. Request DTO được validate và chuẩn hóa.
18. Quota service thực hiện atomic consume.
19. Tạo AI job và idempotency record.
20. Worker gọi AI Orchestrator.
21. Kết quả được validate lại trong Java.
22. Lưu preview; chỉ commit itinerary sau xác nhận của người dùng.

12.3 Error contract

Mã	HTTP	Ý nghĩa
AI_RATE_LIMITED	429	Hết quota, có retryAfter.
AI_BUSY	503	Hàng đợi đầy.
AI_TIMEOUT	504	Model quá thời gian.
AI_INVALID_OUTPUT	502	Model trả dữ liệu không hợp lệ.
AI_RETRIEVAL_EMPTY	422	Không đủ dữ liệu đáng tin cậy.
AI_JOB_CANCELLED	409	Job đã bị hủy.

13 Hàng đợi và streaming

Sinh lịch trình có thể kéo dài hơn request web thông thường. Nên dùng job bất đồng bộ thay vì giữ connection đến khi model kết thúc.

```
POST /trips/{id}/ai/jobs  -> 202 + jobId
GET  /ai/jobs/{jobId}        -> QUEUED | RUNNING | COMPLETED | FAILED
DELETE /ai/jobs/{jobId}      -> yêu cầu hủy
GET  /ai/jobs/{jobId}/events -> SSE cho tiến trình
```

- Queue giới hạn số job chạy đồng thời theo GPU.
- Admission control trả AI_BUSY khi queue vượt ngưỡng.

- Idempotency ngăn double click tạo nhiều job.
- Timeout và cancellation phải giải phóng slot inference.

14 Bảo mật và quyền riêng tư

14.1 Mô hình đe dọa

Rủi ro	Ví dụ	Biện pháp
Prompt injection	Tên chuyến đi yêu cầu bỏ qua luật.	Tách system và data, tool allowlist, validate output.
Cross trip leakage	RAG lấy dữ liệu chuyến đi khác.	Filter bằng tenant và trip scope trước retrieval.
SSRF	Model gọi URL do người dùng cung cấp.	Không có generic HTTP tool.
Denial of wallet	Spam inference làm quá tải GPU.	Rate limit, quota, queue và max tokens.
Sensitive logging	Prompt chứa email hoặc tọa độ.	Redaction và log metadata thay vì nội dung.
Model endpoint exposure	Internet gọi thẳng vLLM.	Private network và reverse proxy.

14.2 Dữ liệu gửi vào model

- Chỉ gửi dữ liệu tối thiểu cần cho nhiệm vụ.
- Không gửi access token, refresh token, mật khẩu hoặc service secret.
- Không gửi vị trí thời gian thực nếu lập lịch trình không cần.
- Gắn retention policy cho prompt và response.
- Cho phép người dùng xóa lịch sử AI liên quan tài khoản.

14.3 Network

- Model server không có public ingress.
- Chặn outbound Internet từ model container nếu không cần.
- Chỉ Orchestrator được phép gọi model server.
- Bảo vệ cả endpoint không nằm dưới /v1 bằng reverse proxy.

15 Quan sát và kiểm soát chi phí

15.1 Metrics

Nhóm	Metric cần có
Traffic	requests, jobs, concurrency, queue depth.
Latency	TTFT, total latency, retrieval latency, validation latency.
Tokens	input, output, cached tokens, tokens per second.

Nhóm	Metric cần có
Quality	schema failures, repair rate, fallback rate, user acceptance.
Resources	GPU utilization, VRAM, power, temperature, OOM.
Reliability	timeouts, cancellations, provider errors, circuit state.

15.2 Log và tracing

Mỗi request cần request ID, job ID, user hash, trip ID hash, model revision, prompt version, retrieval version và timing. Không log raw prompt mặc định. Trace phải nối từ client qua Spring Boot, Orchestrator, retrieval và model server.

15.3 Tính giá nội bộ

Self host không miễn phí. Chi phí gồm GPU, điện, storage, bandwidth, nhân sự vận hành và công suất nhàn rỗi. Theo dõi chi phí trên mỗi job thành công, không chỉ chi phí trên mỗi token.

16 Bộ đánh giá chất lượng

16.1 Dataset

Tạo 200 đến 500 case tiếng Việt, có kết quả mong đợi hoặc rubric. Chia thành development set và locked test set để tránh tối ưu quá mức.

- Chuyến đi 1 đến 7 ngày và nhiều mức ngân sách.
- Gia đình, trẻ nhỏ, người cao tuổi và nhu cầu accessibility.
- Mùa mưa, địa điểm đóng cửa, dữ liệu thiếu hoặc mâu thuẫn.
- Prompt injection, nội dung độc hại và yêu cầu truy cập chuyến đi khác.
- Tọa độ sai, giá âm, destination không tồn tại và output quá dài.

16.2 Chỉ số

Chỉ số	Ngưỡng khởi điểm đề xuất
JSON schema pass rate	Tối thiểu 99 phần trăm sau tối đa một repair.
Grounded destination rate	100 phần trăm destination ID tồn tại và PUBLISHED.
Budget violation	0 phần trăm sau backend validation.
Cross trip leakage	0 case.
P95 latency	Đặt theo UX thực tế sau benchmark.
User acceptance	Theo dõi tỷ lệ lưu hoặc chỉnh sửa kế hoạch.

16.3 Shadow traffic

Trong giai đoạn chuyển đổi, gửi cùng request đã ẩn danh sang Gemini và local model nhưng chỉ trả một kết quả cho người dùng. So sánh tự động và review mẫu. Không lưu hai bộ dữ liệu nhạy cảm lâu hơn cần thiết.

17 Fine tuning và LoRA

17.1 Khi nào cần

Chỉ fine tune khi RAG và prompt đã ổn nhưng model vẫn sai về phong cách, format hoặc quy trình lặp lại. Không fine tune để ghi nhớ giá vé hoặc giờ mở cửa.

17.2 Dữ liệu huấn luyện

- Chỉ dùng dữ liệu có quyền sử dụng.
- Loại bỏ PII và dữ liệu chuyến đi riêng tư.
- Dùng ví dụ đã được chuyên gia duyệt.
- Giữ test set tách biệt tuyệt đối.
- Theo dõi dataset version, base model và adapter checksum.

17.3 Quy trình LoRA

23. Xác định lỗi hành vi lặp lại bằng evaluation.
24. Chuẩn bị instruction response pairs chất lượng cao.
25. Train adapter LoRA trên base model đã pin.
26. So sánh với base model trên locked test set.
27. Kiểm tra regression, safety và tiếng Việt.
28. Canary rollout; có khả năng rollback adapter.

18 Quy trình triển khai từng giai đoạn

Giai đoạn 0 Chuẩn bị

- Định nghĩa SLO, budget, privacy policy và tiêu chí thay Gemini.
- Tạo evaluation dataset ban đầu.
- Thêm AiProvider abstraction và telemetry.

Giai đoạn 1 Proof of concept

- Dựng llama.cpp và Qwen3 8B GGUF trên máy phát triển.
- Kết nối qua OpenAI compatible endpoint.
- Chạy 100 prompt, đo chất lượng và latency.

Giai đoạn 2 RAG

- Cài pgvector, tạo schema knowledge và ingestion job.
- Dựng BGE M3 embedding service.
- Triển khai hybrid retrieval và citations.

Giai đoạn 3 Production serving

- Dựng vLLM trên GPU server private.
- Thêm reverse proxy, queue, quota, circuit breaker và metrics.
- Load test, chaos test và kiểm tra OOM recovery.

Giai đoạn 4 Chuyển traffic

- Shadow traffic rồi canary 5 phần trăm.
- Tăng 10, 25, 50 và 100 phần trăm khi metric đạt ngưỡng.
- Giữ Gemini fallback có thời hạn và alert mỗi lần sử dụng.

Giai đoạn 5 Tối ưu

- Prompt caching, quantization và batching.
- Chỉ cân nhắc LoRA sau khi có dữ liệu thực tế.

19 Runbook vận hành

19.1 Model server OOM

29. Ngừng nhận job mới và mở circuit breaker.
30. Giữ job đang chờ, trả trạng thái AI_BUSY.
31. Thu thập VRAM, batch, context và request gây lỗi.
32. Restart có kiểm soát; không retry vô hạn.
33. Giảm concurrency hoặc context trước khi mở traffic.

19.2 Chất lượng giảm sau nâng model

34. Dừng rollout và quay về model revision trước.
35. So sánh prompt version, tokenizer, chat template và quantization.
36. Chạy locked regression suite.
37. Chỉ rollout lại sau khi có báo cáo nguyên nhân.

19.3 RAG trả dữ liệu sai

38. Tắt index hoặc source revision lỗi.
39. Kiểm tra filter PUBLISHED, tenant scope và content hash.
40. Reindex có version; không overwrite mù.
41. Kiểm tra citation trước khi mở lại.

19.4 Key hoặc model bị xâm phạm

42. Cô lập model service và thu hồi service token.
43. Chặn network, bảo toàn log metadata và xác định phạm vi.
44. Thay weights từ checksum tin cậy và xoay secret.
45. Thông báo theo incident policy nếu có dữ liệu người dùng.

20 Checklist nghiệm thu

Kiến trúc

- ☐ AI Provider abstraction hoạt động.
- ☐ Model server private và không public ingress.
- ☐ RAG chỉ dùng dữ liệu PUBLISHED.
- ☐ Backend là nơi validation và lưu nghiệp vụ.

Bảo mật

- ☐ Không log prompt chứa PII.
- ☐ Không có generic SQL hoặc HTTP tool.
- ☐ Rate limit và quota atomic.
- ☐ Có audit và request tracing.
- ☐ Có test cross trip leakage.

Chất lượng

- ☐ Schema pass đạt ngưỡng.
- ☐ Không destination bịa.
- ☐ Không vượt ngân sách sau validation.
- ☐ Có citations.
- ☐ Locked evaluation suite chạy trong CI.

Vận hành

- ☐ Có dashboard GPU và queue.
- ☐ Có timeout, cancellation và circuit breaker.
- ☐ Có canary và rollback.
- ☐ Weights, prompt, embedding đều có version.
- ☐ Runbook đã diễn tập.

Phụ lục A Cấu hình khởi đầu đề xuất

```
Model: Qwen3 8B
Development: llama.cpp với GGUF Q4 hoặc Q5
Production: vLLM trên GPU server private
Embedding: BGE M3
Vector store: PostgreSQL và pgvector
Queue và quota: Redis
Orchestration: service riêng
Business validation: Spring Boot
Client progress: job polling hoặc SSE
```

Phụ lục B Câu hỏi cần trả lời trước khi mua hạ tầng

46. Bao nhiêu request AI mỗi ngày và vào giờ cao điểm?
47. Context và output trung bình bao nhiêu token?
48. P95 latency chấp nhận được là bao nhiêu?
49. Có bao nhiêu request đồng thời?

50. Dữ liệu nào được phép đi vào model?
51. SLO và thời gian khôi phục là bao nhiêu?
52. Ngân sách GPU theo tháng là bao nhiêu?
53. Ngưỡng chất lượng nào cho phép tắt Gemini?

Phụ lục C Nguồn tham khảo chính

Qwen3 official announcement: https://qwenlm.github.io/blog/qwen3/

Qwen3 8B model card: https://huggingface.co/Qwen/Qwen3-8B

vLLM documentation: https://docs.vllm.ai/en/stable/

vLLM OpenAI compatible server and security note: https://github.com/vllm-project/vllm/blob/main/docs/serving/online_serving/openai_compatible_server.md

llama.cpp official repository: https://github.com/ggml-org/llama.cpp

BGE M3 model card: https://huggingface.co/BAAI/bge-m3

pgvector official repository: https://github.com/pgvector/pgvector